

Глава 11. Визуализация модели линейной регрессии

Инструменты этой группы обладают очень интересным функционалом: они строят модель линейной регрессии по переданным данным и отображают ее вместе с исходным набором. Функции представлены в таблице 11.1.

Таблица 11.1 — Функции для визуализации модели линейной регрессии

Функция	Описание
<code>lmpplot()</code>	Отображает набор данных и линию регрессии, построенную по ним, с возможностью управлять компоновкой полей с графиками на подложке.
<code>regplot()</code>	Отображает набор данных и линию регрессии, построенную по ним.
<code>residplot()</code>	Отображает отклонения элементов исходного набора данных от регрессионной модели, построенной по ним, в виде диаграммы рассеяния.

11.1 Общие параметры функций

Рассмотрим некоторые из общих параметров функций визуализации модели линейной регрессии:

- `x, y`: имена переменных из набора `data`, `optional`
 - Связывают ось `x` и `y` с конкретными признаками из набора данных, переданного через параметр `data`. Для функций, отличных от `catplot()`, допустимо передавать вектора с данными напрямую, для `catplot()` — нет. Параметры могут иметь значение `None`, в случае если необходимо визуализировать весь набор `data`.

- `data: DataFrame`
 - Набор данных в формате `pandas.DataFrame`, в котором столбцы — это признаки, строки — значения. Имена столбцов, данные из которых необходимо визуализировать, передаются в параметры `x` и `y`.
- `lowess: bool, optional`
 - Если параметр равен `True`, то будет использована модель из *statsmodels* для построения локально взвешенной линейной регрессии. Доверительный интервал для такой модели не отображается.
- `robust: bool, optional`
 - Если параметр равен `True`, то будет использована модель из *statsmodels* для оценки робастной регрессии.
- `{scatter,line}_kws: dict`
 - Определяют дополнительные аргументы функций `plt.scatter()` и `plt.plot()`.

11.2 Функция `regplot()`

Функция `regplot()` отображает набор данных и линию регрессии, построенную по ним. Загрузим наборы данных *iris* и *mpg*:

```
mpg = sns.load_dataset("mpg")
iris = sns.load_dataset("iris")
```

Построим линию регрессии зависимости расстояния, которое может проехать автомобиль, от его мощности. Через параметры `x` и `y` передаём названия признаков, через `data` — набор данных.

```
sns.lmplot(x="horsepower", y="displacement", data=mpg)
```

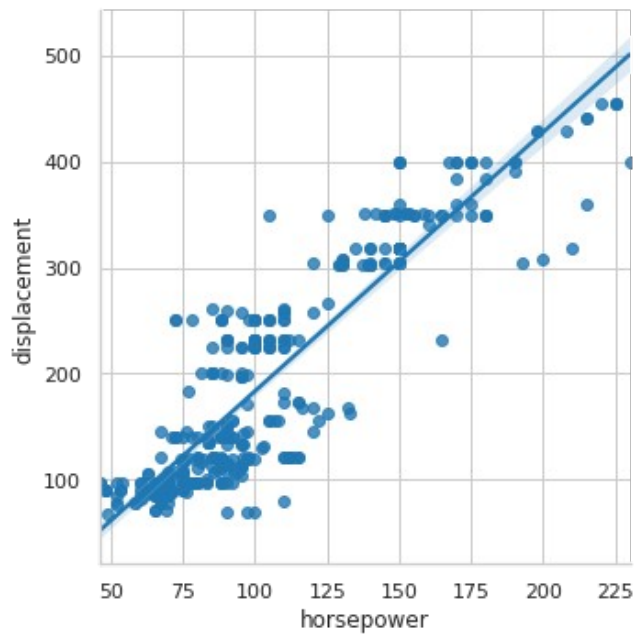


Рисунок 11.1 — Демонстрация работы функции `regplot()`

Из набора *iris* выберем только те ирисы, которые относятся к классу *setosa*:

```
iris_mod = iris[iris["species"] == "setosa"]
```

Построим зависимость ширины от длины наружной доли околоцветника:

```
sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod)
```

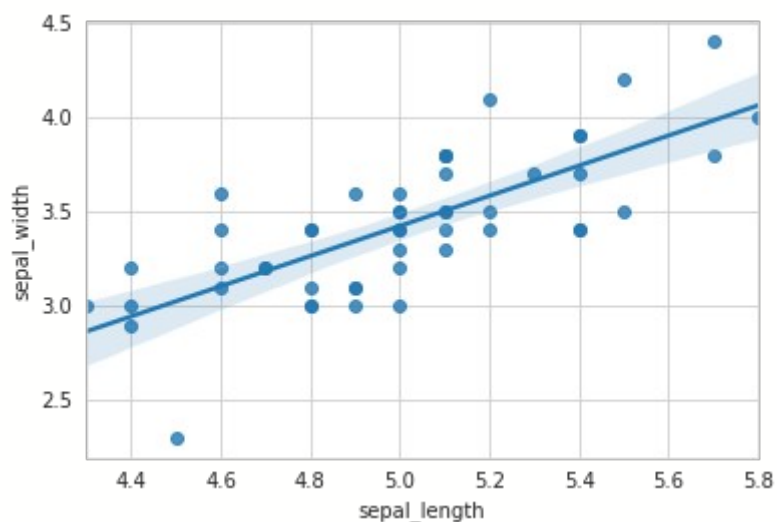


Рисунок 11.2 — Визуализации зависимости параметров *sepal_length* и *sepal_width* из набора данных *iris*

С поля графика можно убрать либо точки (параметр `scatter`), либо регрессионную прямую (параметр `fit_reg`). Продемонстрируем это, присвоим параметру `scatter` значение `False`:

```
sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod,  
scatter=False)
```

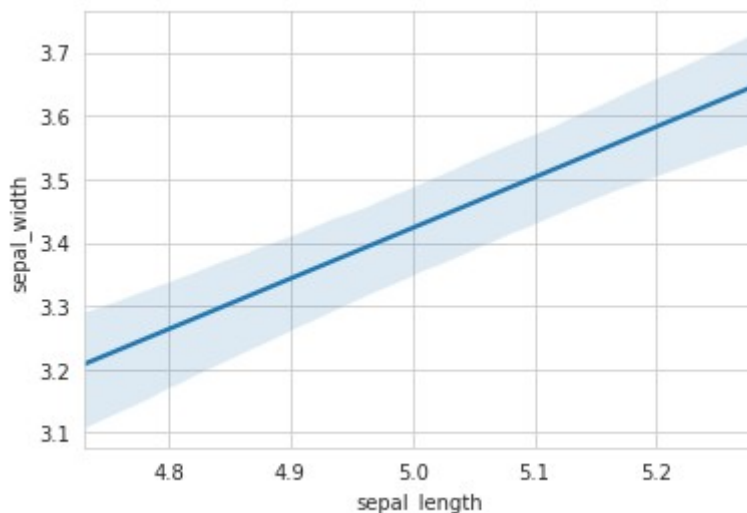


Рисунок 11.3 — Демонстрация работы с параметром `scatter` функции `regplot()`

Зададим параметру `fit_reg` значение `False`:

```
sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod,  
fit_reg=False)
```

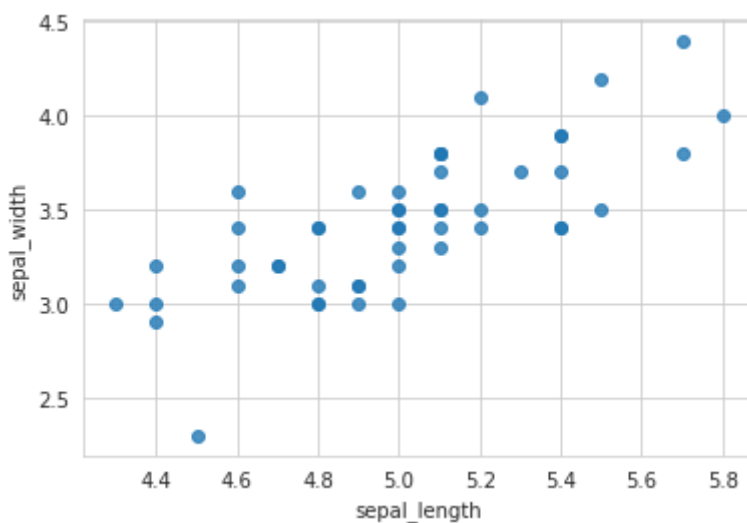


Рисунок 11.4 — Демонстрация работы с параметром `fit_reg` функции `regplot()`

Для настройки внешнего вида графиков можно воспользоваться параметрами:

- `color`: *Matplotlib*-цвет, optional
 - Цвет отображаемых элементов (точки и линия).
- `marker`: код маркера из *Matplotlib*
 - Стилль маркера, которым отображаются точки.

```
sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod, color='r')
```

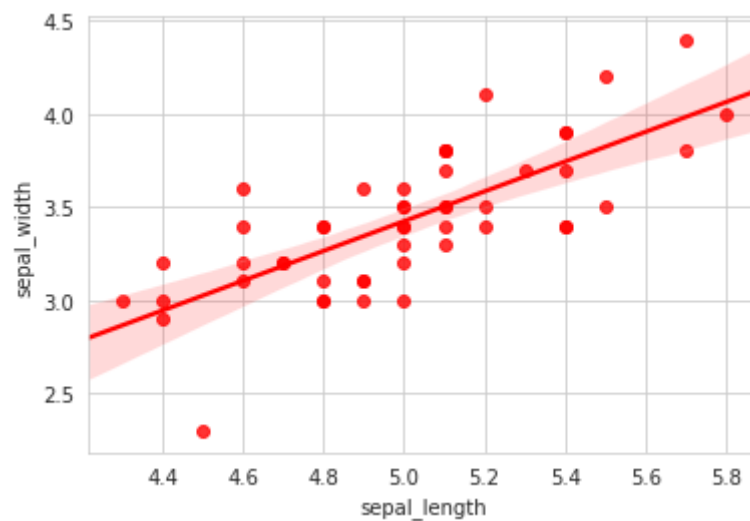


Рисунок 11.5 — Демонстрация работы с параметром `color` функции `regplot()`

```
markers = ["^", "*", "o"]  
plt.figure(figsize=(15, 5))  
for i, m in enumerate(markers):  
    plt.subplot(1, len(markers), i+1)  
    plt.title(f"marker = {m}")  
    sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod,  
color='r', marker=m)
```

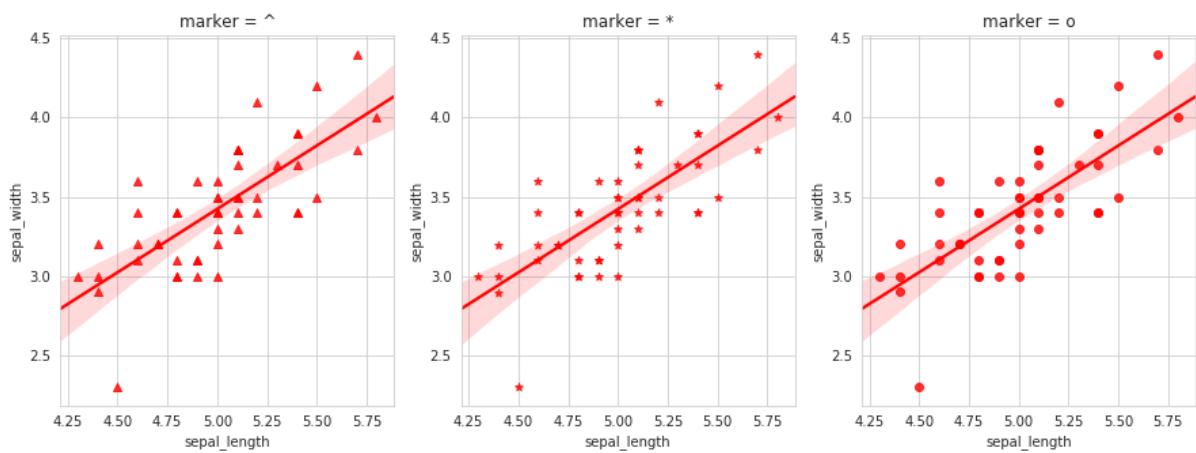


Рисунок 11.6 — Демонстрация работы с параметром `marker` функции `regplot()`

Для более тонкой настройки используйте параметры `scatter_kws`, `line_kws`:

```
s_kws = {"linewidths":2, "edgecolors":'g', 'color':'r', "s": 50}
p_kws = {'ls':'-.', 'lw':3}
sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod,
scatter_kws=s_kws, line_kws=p_kws)
```

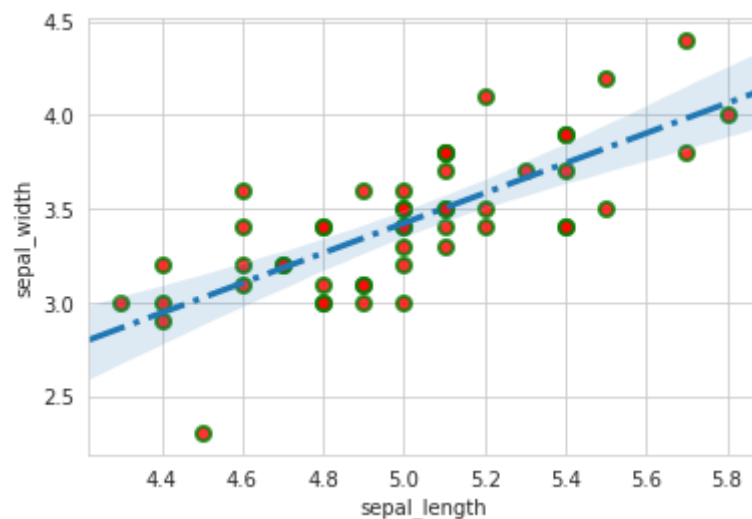


Рисунок 11.7 — Демонстрация работы с параметром `scatter_kws` и `line_kws` функции `regplot()`

Функция `regplot()` предоставляет альтернативу точечной диаграмме — диаграмму со значениями оценок и доверительными интервалами. Для настройки этого режима используются параметры:

- `x_estimator`: функция, optional
 - Функция для вычисления значения оценки.
- `x_bins`: `int` или `vector`, optional
 - Определяет количество групп, на которые будет разбито исходное множество значений.
- `x_ci`: `'ci'`, `'sd'`, `int` в диапазоне `[0, 100]` или `None`, optional
 - Размер доверительного интервала. Если значение равно `'sd'`, то вместо доверительного интервала будет использоваться стандартное отклонение.

Построим диаграмму с медианной оценкой:

```
from numpy import median  
sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod,  
x_estimator=median)
```

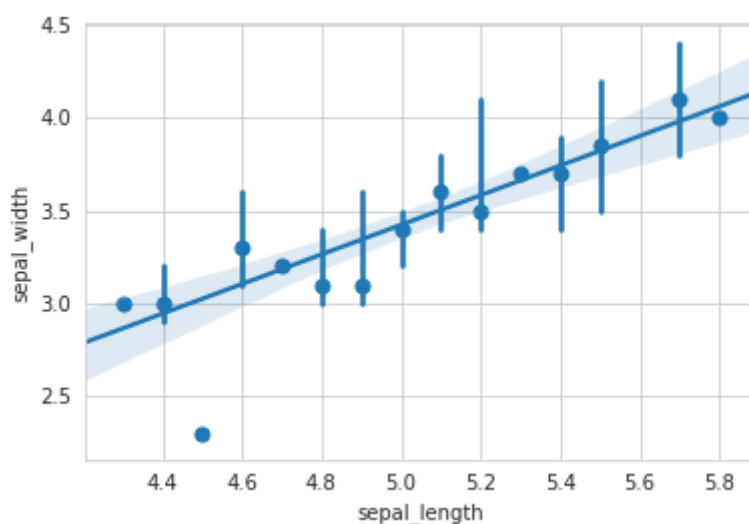


Рисунок 11.8 — Диаграмма с медианной оценкой

Пример работы с параметром `x_bins`:

```
x_bins = [5, 10, None]
plt.figure(figsize=(15, 5))
for i, b in enumerate(x_bins):
    plt.subplot(1, len(x_bins), i+1)
    plt.title(f"x_bins = {b}")
    sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod,
x_bins=b)
```

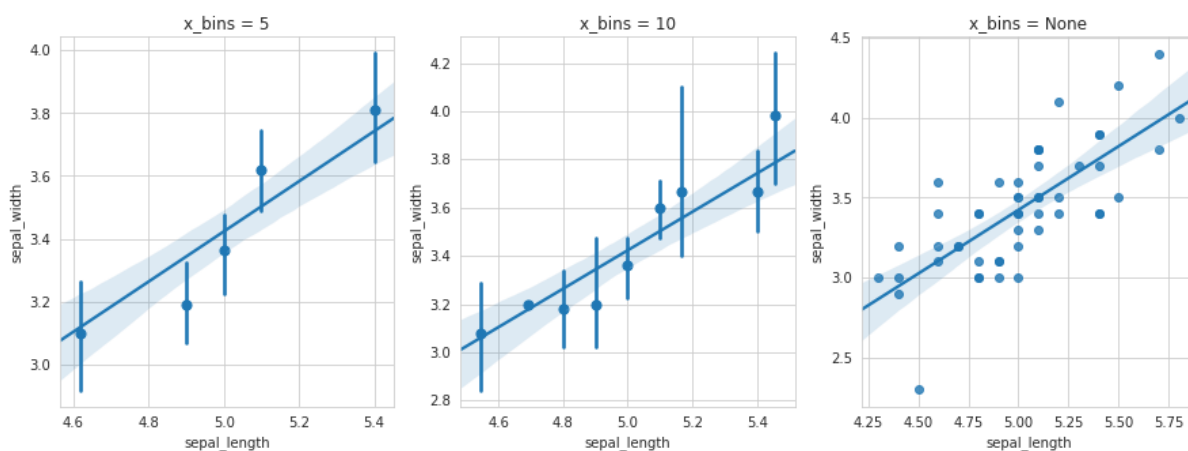


Рисунок 11.9 — Демонстрация работы с параметром `x_bins` функции `regplot()`

Пример работы с параметром `x_ci`:

```
x_ci = [50, 80, "sd"]
plt.figure(figsize=(15, 5))
for i, xc in enumerate(x_ci):
    plt.subplot(1, len(x_ci), i+1)
    plt.title(f"x_ci = {xc}")
    sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod,
x_ci=xc, x_bins=10)
```

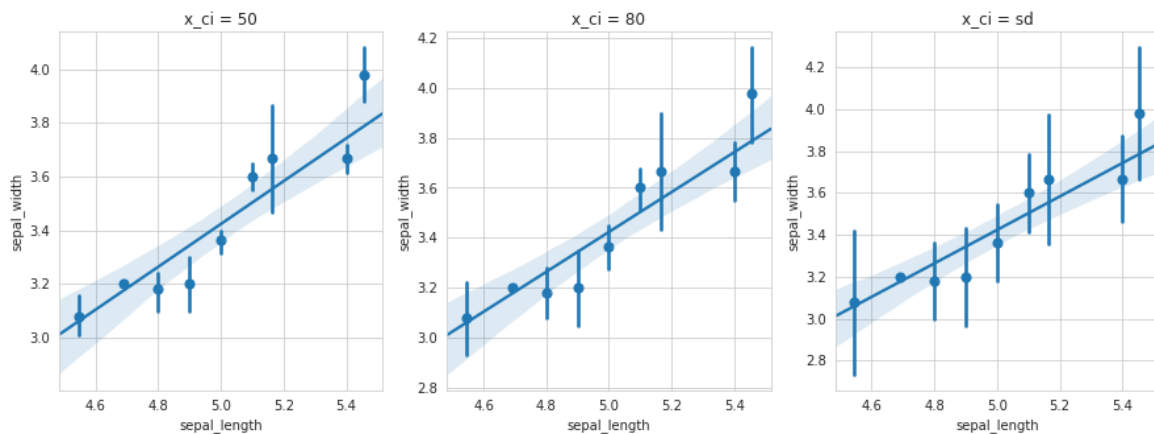


Рисунок 11.10 — Демонстрация работы с параметром `x_ci` функции `regplot()`

Как вы уже могли заметить, вокруг линии регрессии есть область того же цвета, что и основная линия, но более светлого оттенка. Она представляет доверительный интервал, расчётом этой области управляют следующие параметры:

- `ci`: `int` в диапазоне `[0, 100]` или `None`, `optional`
 - Размер доверительного интервала.
- `n_boot`: `int`, `optional`
 - Количество образцов для оценки `ci`.

Посмотрим как будет выглядеть диаграмма с различными значениями `ci`:

```
ci = [50, 99, None]
plt.figure(figsize=(15, 5))
for i, c in enumerate(ci):
    plt.subplot(1, len(ci), i+1)
    plt.title(f"ci = {c}")
    sns.regplot(x="sepal_length", y="sepal_width", data=iris_mod, ci=c)
```

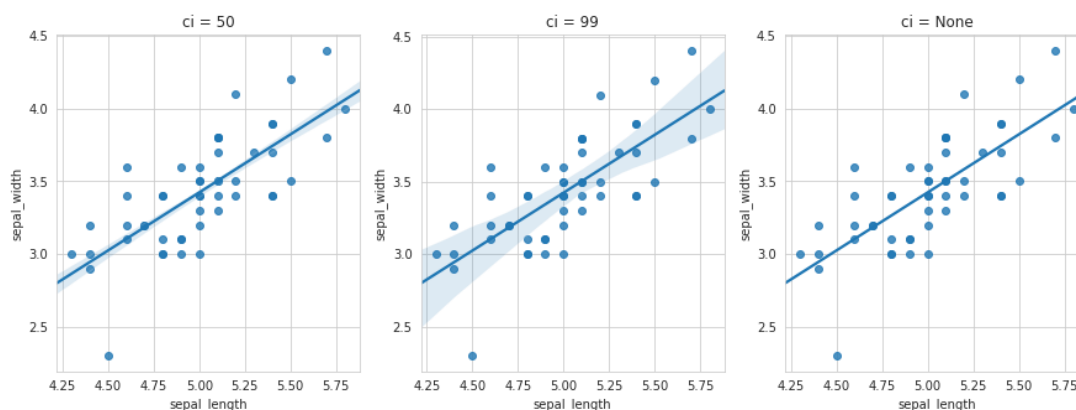


Рисунок 11.11 — Демонстрация работы с параметром `ci` функции `regplot()`

Рассмотрим некоторые из параметров функции `regplot()`:

- `logistic: bool, optional`
 - Если параметр равен `True`, то будет принято предположение, что `y` является бинарной переменной и для построения модели логистической регрессии будет использоваться модель из `statsmodels`.

Добавим в набор данных `iris` ещё один признак: `is_setosa`, который принимает значение `True`, если ирис относится к классу `setosa`, иначе — значение `False`:

```
iris["is_setosa"] = iris["species"] == 'setosa'
```

```
sns.regplot(x="sepal_length", y="is_setosa", logistic=True, data=iris)
```

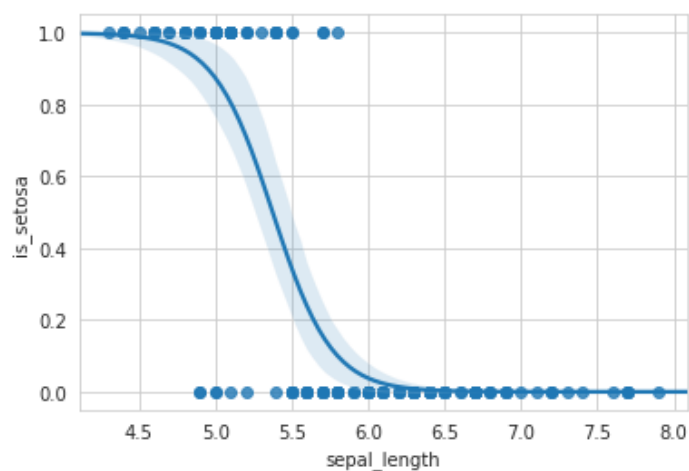


Рисунок 11.12 — Демонстрация работы с параметром `logistic` функции `regplot()`

- `robust: bool, optional`
 - Если параметр равен `True`, то будет использована модель из `statsmodels` для оценки робастной регрессии.

Пример работы с параметром `robust`:

```
sns.regplot(x="sepal_length", y="sepal_width", robust=True,
data=iris_mod)
```

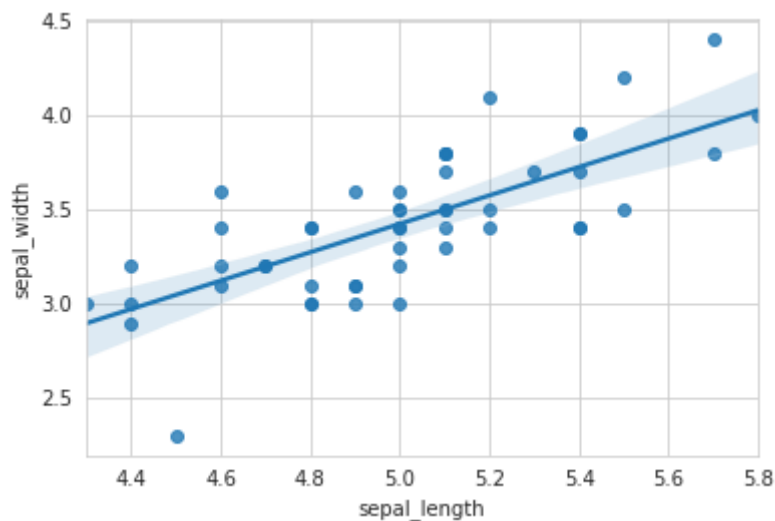


Рисунок 11.13 — Демонстрация работы с параметром `robust` функции `regplot()`

- `logx: bool, optional`
 - Если значение равно `True`, то оценивается линейная регрессия вида $y \sim \log(x)$. Значения параметра x должны быть больше нуля.

Пример работы с параметром `logx`:

```
sns.regplot(x="mpg", y="displacement", logx=True, data=mpg)
```

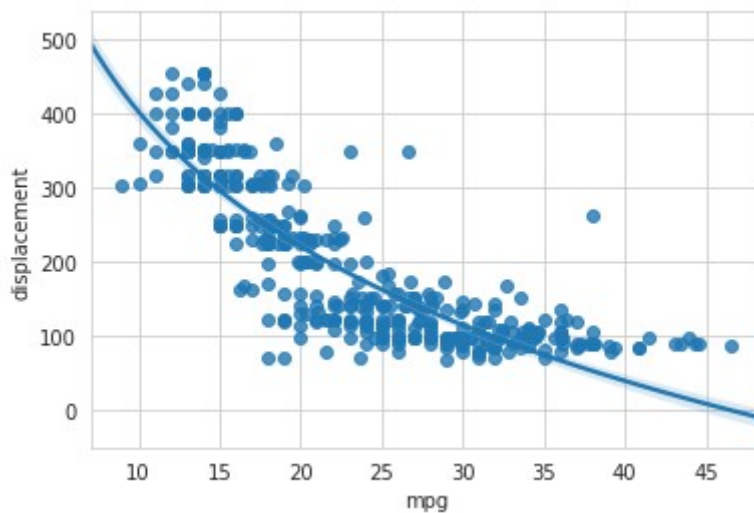


Рисунок 11.14 — Демонстрация работы с параметром `logx` функции `regplot()`

- `lowess: bool, optional`
 - Если параметр равен `True`, то будет построена локально взвешенная линейная регрессия. Доверительный интервал для такой модели не отображается.

Пример работы с параметром `lowess`:

```
sns.regplot(x="mpg", y="displacement", lowess=True, data=mpg)
```

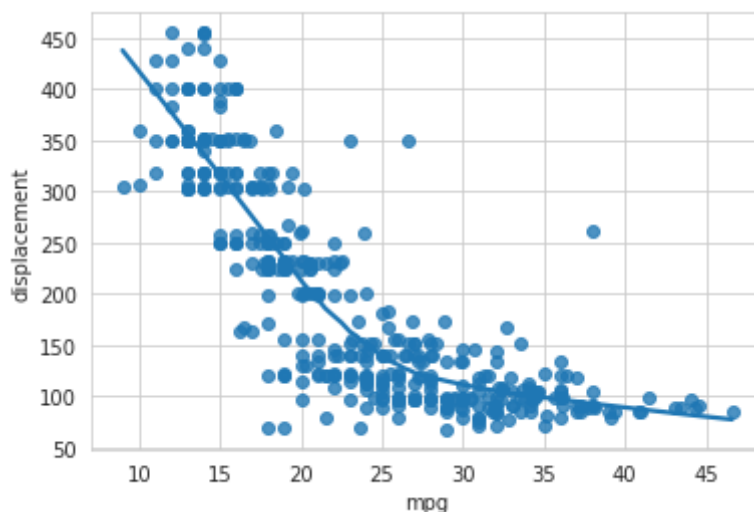


Рисунок 11.15 — Демонстрация работы с параметром `lowess` функции `regplot()`

- `order: int, optional`
 - Если значение параметра больше 1, то будет использована функция `numpy.polyfit` для построения полиномиальной регрессии.

Пример работы с параметром `order`:

```
order=[1, 2, 3]
plt.figure(figsize=(15, 5))
for i, o in enumerate(order):
    plt.subplot(1, len(order), i+1)
    plt.title(f"order = {o}")
    sns.regplot(x="mpg", y="displacement", order=o, data=mpg)
```

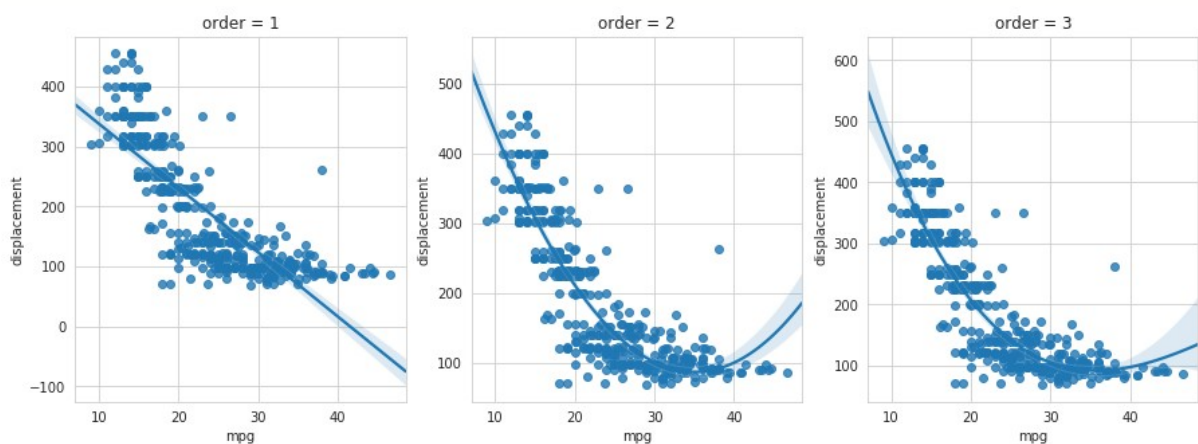


Рисунок 11.16 — Демонстрация работы с параметром `order` функции `regplot()`

- `truncate: bool, optional`
 - По умолчанию линия регрессии отображается для всего диапазона оси `x`. Если параметр равен `True`, то линия будет ограничена крайними значениями набора данных.

Пример работы с параметром `truncate`:

```
sns.regplot(x="mpg", y="displacement", truncate=True, data=mpg)
```

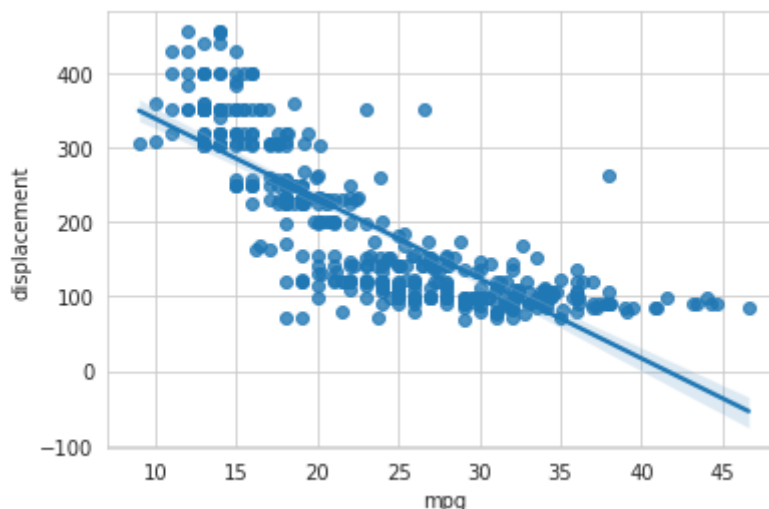


Рисунок 11.17 — Демонстрация работы с параметром `truncate` функции `regplot()`

11.3 Функция `residplot()`

Функция `residplot()` отображает отклонения элементов исходного набора данных от регрессионной модели, построенной по ним, в виде диаграммы рассеяния. Каждая точка такой диаграммы — это разность между значением элемента исходного набора и значением, которое выдаст модель регрессии в этой точке. Параметры функции `residplot()` совпадают с набором, приведённым в разделе "11.1 Общие параметры функций".

Приведём несколько примеров её использования. Будем работать с набором `mpg`:

```
mpg = sns.load_dataset("mpg")
```

Построим модель линейной регрессии с помощью функции `regplot()`:

```
sns.regplot(x="mpg", y="displacement", data=mpg)
```

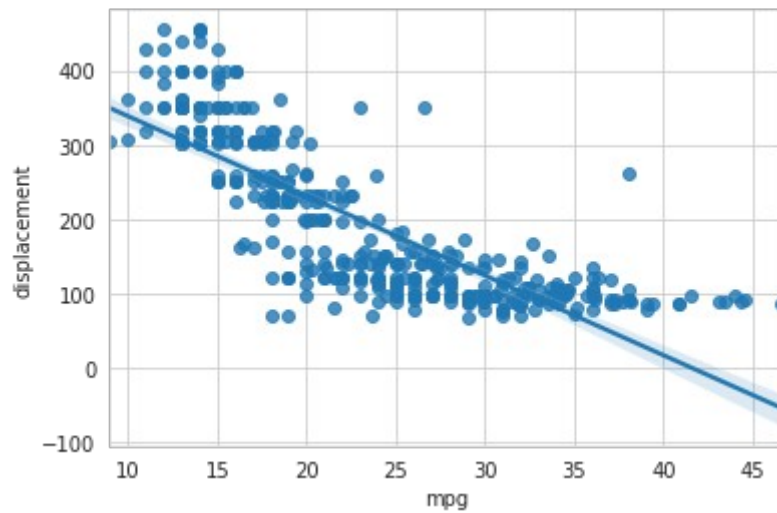


Рисунок 11.18 — График модели линейной регрессии, построенный с помощью `regplot()`

Теперь построим диаграмму рассеяния отклонений значений набора данных от модели:

```
sns.residplot(x="mpg", y="displacement", color='g', data=mpg)
```

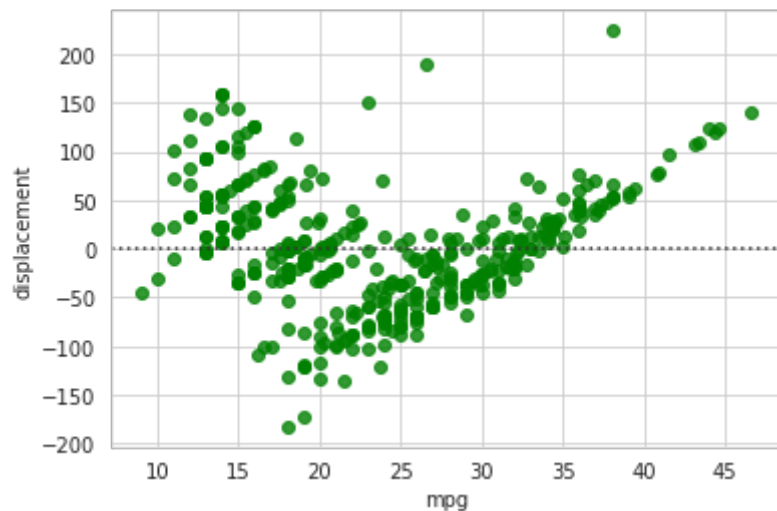


Рисунок 11.19 — Диаграмма рассеяния отклонений значений набора данных от модели, построенная с помощью `residplot()`

Пример модели третьего порядка:

```
sns.regplot(x="mpg", y="displacement", order=3, data=mpg)
```

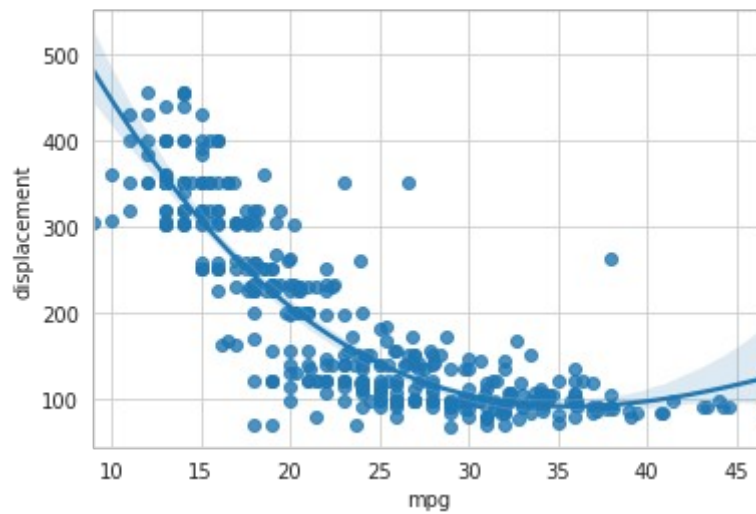


Рисунок 11.20 — График модели третьего порядка

Для такой модели распределение отклонений будет таким:

```
sns.residplot(x="mpg", y="displacement", order=3, color='g', data=mpg)
```

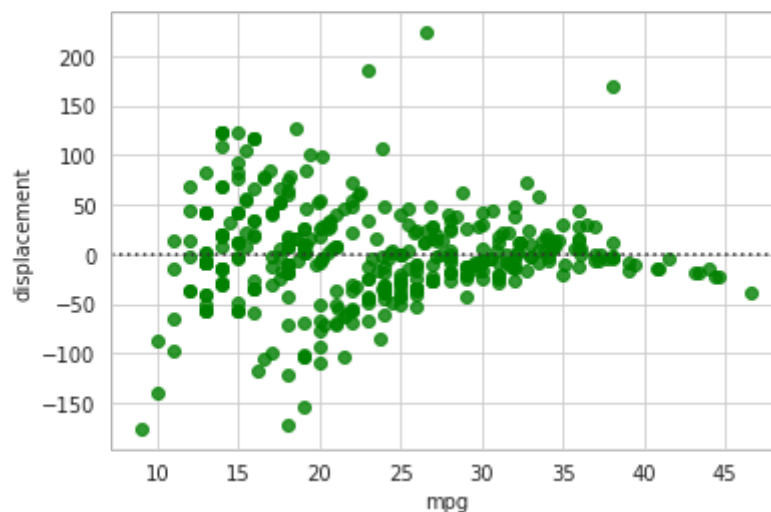


Рисунок 11.21 — Распределение отклонений для модели третьего порядка

11.4 Функция `lmpplot()`

Функция `lmpplot()` по своему назначению аналогична `regplot()` (см. раздел "11.2 Функция `regplot()`") с возможностью управлять компоновкой полей с графиками на подложке.

Загрузим наборы данных *mpg* и *iris*:

```
mpg = sns.load_dataset("mpg")
iris = sns.load_dataset("iris")
```

Если пока оставить в стороне параметры, отвечающие на настройку подложки, то в остальном, аргументы функции `lmpplot()` практически полностью совпадают по названию и назначению с рассмотренными для функции `regplot()`. К ним относятся `x`, `y`, `data`, `order`, `x_bins`, `units`, `ci`, `x_ci`, `n_boot`, `{x,y}_jitter`, `{x, y}_partial`, `x_estimator`, `scatter`, `fit_reg`, `logistic`, `robust`, `logx`, `lowess`, `truncate`, `{scatter, line}_kws`, `dropna`.

Параметры, отвечающие за разделение данных на группы и управление отображением диаграмм функции `lmpplot()` совпадают с аргументами функций `relplot()` и `catplot()`, к ним относятся `hue`, `row`, `col`, `pallette`, `hue_order`, `legend`, `legend_out`, `height`, `aspect`. Поэтому мы не будем приводить полное их описание, ограничимся примером работы с ними.

Построим график с моделью регрессии с помощью функции `lmpplot()`, такой же как мы могли бы получить с помощью функции `regplot()`.

```
sns.lmpplot(x="horsepower", y="displacement", data=mpg)
```

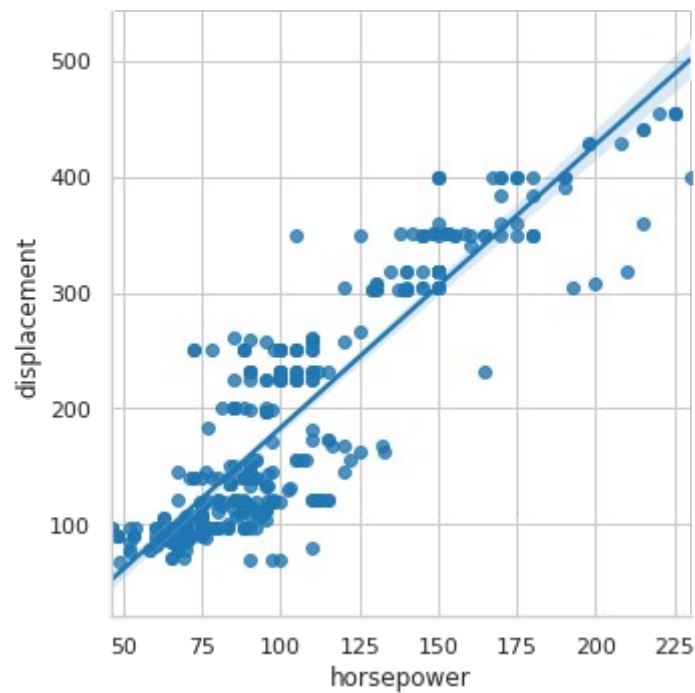


Рисунок 11.22 — График модели линейной регрессии, построенный с помощью `lmplot()`

Для разделения данных по категориальному признаку с последующим построением для каждой группы своей модели регрессии в функции `lmplot()` используются параметры:

- `hue`: для цветового разделения;
- `row`: для вертикального представления диаграмм;
- `col`: для горизонтального представления диаграмм.

За настройку цветовой схемы отвечает параметр `pallette`.

Построим диаграмму для набора `iris` с разделением по типу ириса (признак `species`):

```
sns.lmplot(x="sepal_width", y="petal_width", hue="species", data=iris)
```

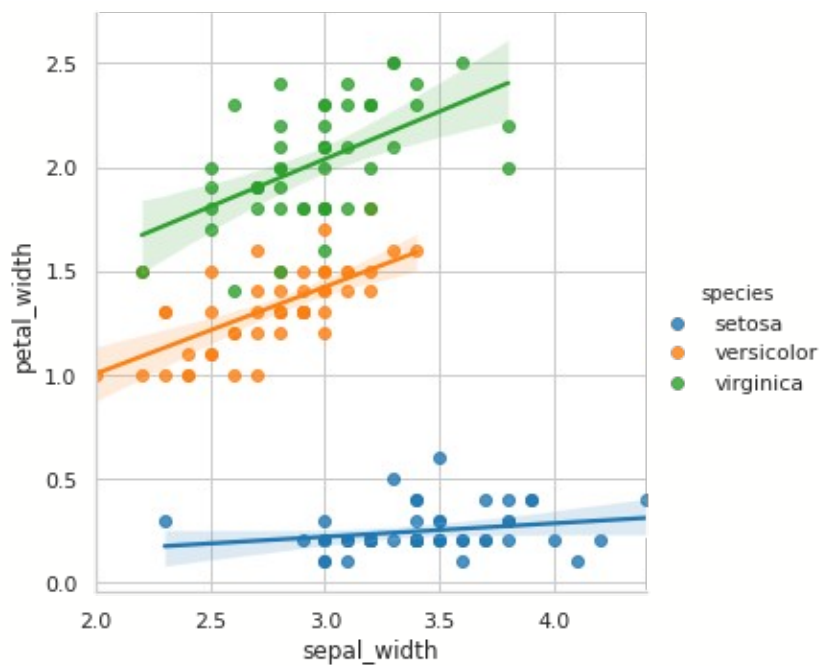


Рисунок 11.23 — Демонстрация работы с параметром hue функции lplot()

Изменим палитру:

```
sns.lplot(x="sepal_width", y="petal_width", hue="species",
palette="Dark2", data=iris)
```

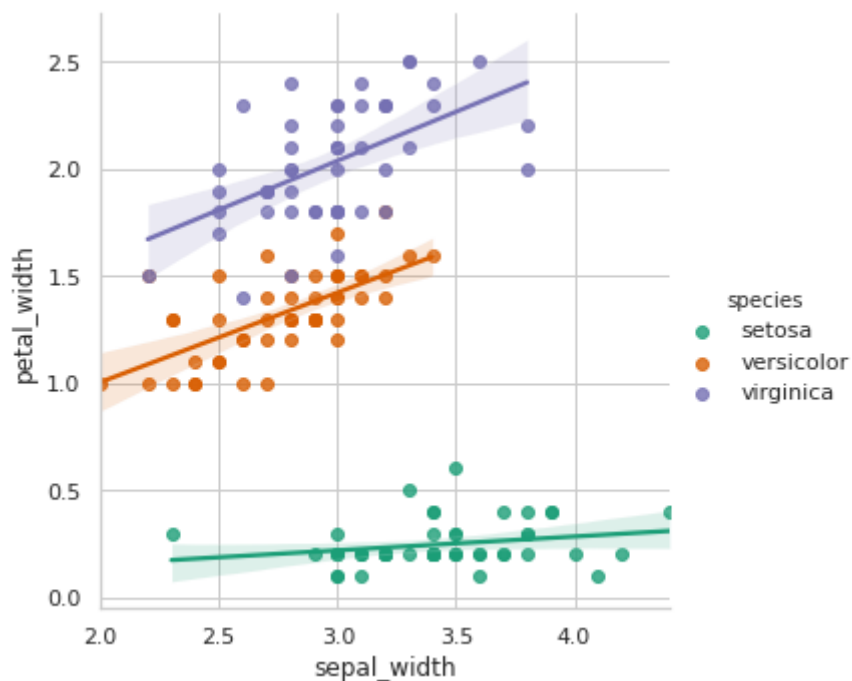


Рисунок 11.24 — Демонстрация работы с параметром palette функции lplot()

Представим графики для каждого отдельного значения *species* на разных полях с горизонтальным разделением:

```
sns.lmplot(x="sepal_length", y="sepal_width", hue="species",  
palette="Dark2", col="species", data=iris)
```

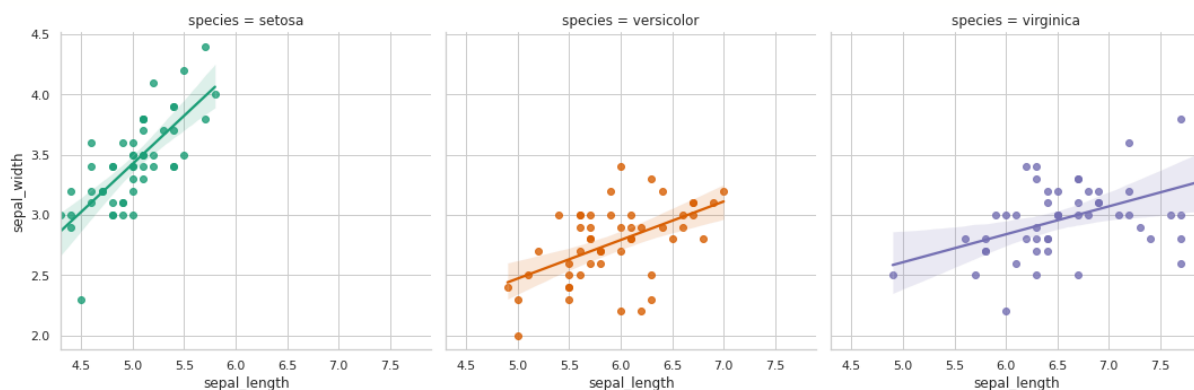


Рисунок 11.25 — Демонстрация работы с параметром col функции lmplot()

С вертикальным разделением:

```
sns.lmplot(x="sepal_length", y="sepal_width", hue="species",  
palette="Dark2", row="species", data=iris)
```

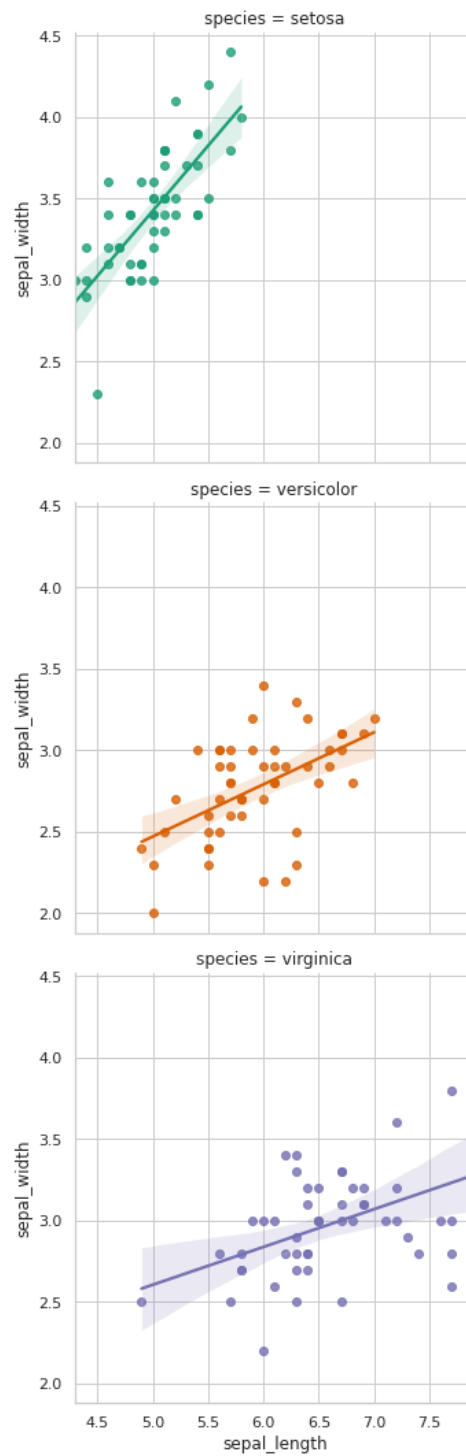


Рисунок 11.26 — Демонстрация работы с параметром row функции `lplot()`

Для управления размером диаграмм используйте параметры `height` (высота) и `aspect` (соотношение сторон):

```
sns.lmplot(x="sepal_length", y="sepal_width", hue="species",  
palette="Dark2", col="species", height=3, aspect=1.5, data=iris)
```

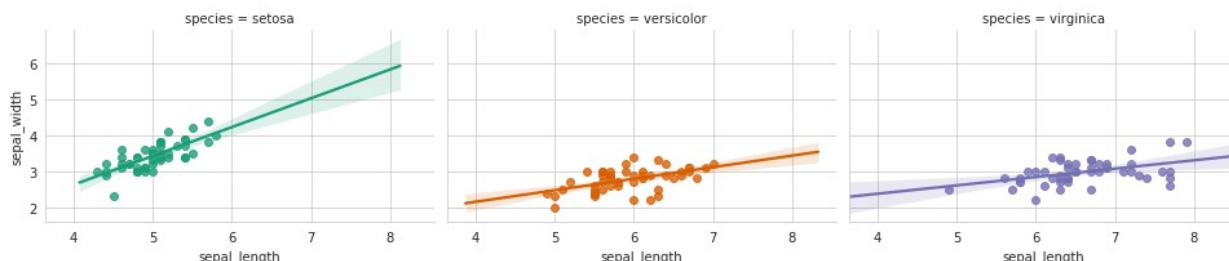


Рисунок 11.27 — Демонстрация работы с параметром `height` и `aspect` функции `lmplot()`

Параметры `sharex` и `sharey` отвечают за отображение осей `x` и `y` на всех диаграммах либо только на крайней левой (при горизонтальном расположении) или на нижней (при вертикальном расположении):

```
sns.lmplot(x="sepal_length", y="sepal_width", col="species",  
sharey='col', data=iris)
```

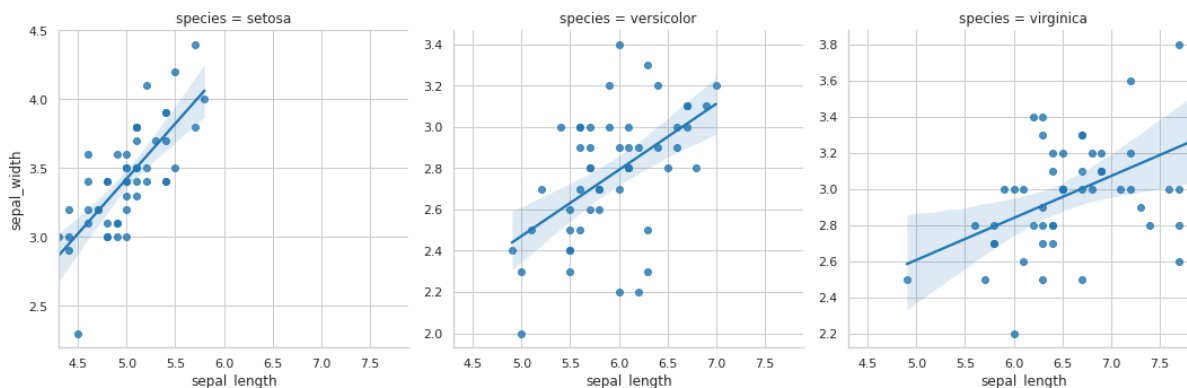


Рисунок 11.28 — Демонстрация работы с параметром `sharey` функции `lmplot()`

Порядок задания цвета (если вы используете параметр `hue`) или порядок вывода диаграмм (если для разделения на группы используется `col` или

row) задаётся с помощью `hue_order`, `col_order`, `row_order`. Приведём пример для `hue_order`:

```
h_order = ["virginica", "setosa", "versicolor"]  
sns.lmplot(x="sepal_width", y="petal_width", hue="species",  
hue_order=h_order, data=iris)
```

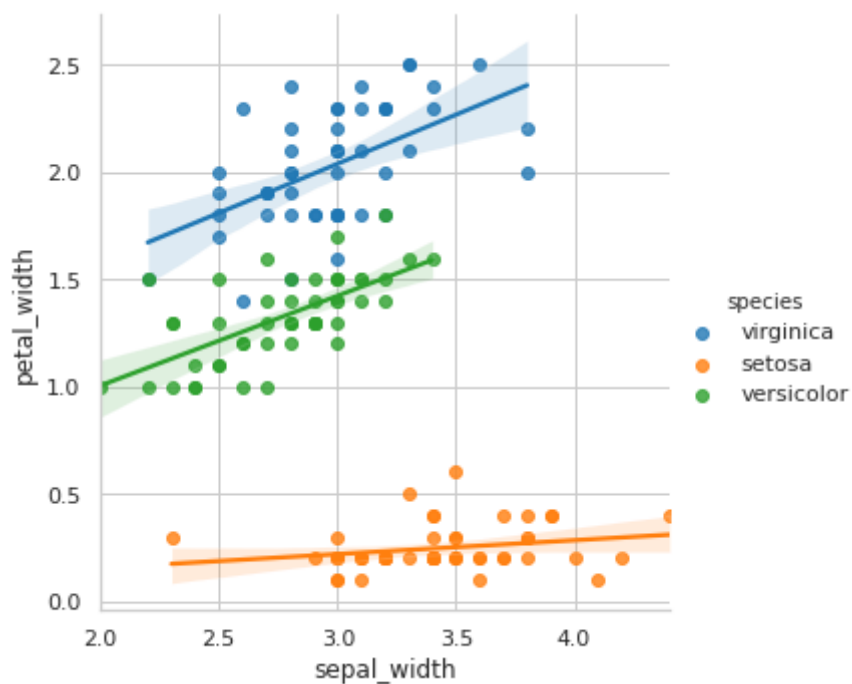


Рисунок 11.29 — Демонстрация работы с параметром `hue_order` функции `lmplot()`

За отображение легенды отвечает параметр `legend`, за вынос её за пределы поля с графиками — `legend_out`.